

Reverse Nodes in K-Group

Complete Personal DSA Notes — intuition, required code comments, dry-run guidance, and clear complexity

Study-note rule: Add comments only where pointer changes, boundary conditions, or variable roles may cause confusion. Explain intuition before code. Explain time and space complexity clearly. If a solution is recursive, include a dry run of recursive calls and backtracking. This K-group solution is iterative, so its dry run focuses on pointer movement and group connections.

1. Problem Statement

Given a singly linked list and an integer k , reverse every complete group of k nodes. If the final group has fewer than k nodes, keep it unchanged.

1 → 2 → 3 | 4 → 5 → 6 | 7 → 8 → 9 | 10

3 → 2 → 1 | 6 → 5 → 4 | 9 → 8 → 7 | 10

2. Intuition

Think: **find one complete K-group** → **isolate it** → **reverse it** → **connect it to the previous group** → **move to the next group**. The main questions are: where does the group start ($temp$), where does it end ($kthNode$), where does the remaining list start ($nextNode$), and where is the previous group's last node ($prevLast$)?

```
Find Kth node
↓
No Kth node → keep remainder unchanged → stop
↓
Save nextNode
↓
Cut kthNode.next = null
↓
Reverse current group
↓
First group? → head = kthNode
Later group? → prevLast.next = kthNode
↓
prevLast = temp
↓
temp = nextNode
```

3. Pointer Roles

temp: first node of current group. **kthNode**: last node of current group. **nextNode**: first node of remaining list; save it before cutting. **prevLast**: last node of previous reversed group. **head**: overall answer head, changed after the first group.

4. Why nextNode Must Be Saved

1 → 2 → 3 → 4 → 5 → 6
 ↑ ↑
 temp kthNode

```
nextNode = kthNode.next; // saves 4
kthNode.next = null;     // cuts 3 → 4
```

After cutting, the current group is independent. Without $nextNode$, the connection to the rest of the list would be lost.

5. getKthNode()

```
private ListNode getKthNode(ListNode temp, int k) {
    ListNode current = temp;

    for (int i = 1; i < k; i++) {
        if (current == null) {
            return null;
        }
        current = current.next;
    }
    return current;
}
```

```

        // No complete K-group remains.
    }
    current = current.next;
}

return current;
// Return the K-th node.
}

```

Intuition: move $k-1$ times from the group's first node. If the list ends before reaching the K -th node, return null.

6. reverseLinkedList()

```

private ListNode reverseLinkedList(ListNode head) {
    ListNode prev = null;
    ListNode current = head;

    while (current != null) {
        ListNode front = current.next;
        // Save next before changing the link.

        current.next = prev;
        // Reverse current node's pointer.

        prev = current;
        current = front;
    }

    return prev;
    // New head after reversal.
}

```

7. Complete Java Code — Required Comments Included

```

class Solution {

    private ListNode getKthNode(ListNode temp, int k) {
        ListNode current = temp;

        for (int i = 1; i < k; i++) {
            if (current == null) {
                return null;
                // No complete K-group remains.
            }
            current = current.next;
        }

        return current;
    }

    private ListNode reverseLinkedList(ListNode head) {
        ListNode prev = null;
        ListNode current = head;

        while (current != null) {
            ListNode front = current.next;
            // Save next before changing the link.

            current.next = prev;
            // Reverse current pointer.

            prev = current;
            current = front;
        }

        return prev;
    }

    public ListNode reverseKGroup(ListNode head, int k) {
        ListNode temp = head;
        // Start of current K-group.

        ListNode prevLast = null;
        // Last node of previous reversed group.
    }
}

```

```

while (temp != null) {
    ListNode kthNode = getKthNode(temp, k);
    // Check for a complete K-group.

    if (kthNode == null) {
        if (prevLast != null) {
            prevLast.next = temp;
            // Attach incomplete remainder unchanged.
        }
        break;
    }

    ListNode nextNode = kthNode.next;
    // IMPORTANT: save next group before cutting.

    kthNode.next = null;
    // Isolate exactly K nodes.

    reverseLinkedList(temp);
    // Reverse only this group.

    if (temp == head) {
        head = kthNode;
        // First group changes overall head.
    } else {
        prevLast.next = kthNode;
        // Later groups connect here.
    }

    prevLast = temp;
    // Original first node becomes last after reversal.

    temp = nextNode;
    // Continue with saved next group.
}

return head;
}
}

```

8. Lines to Remember

```

ListNode kthNode = getKthNode(temp, k);
ListNode nextNode = kthNode.next; // Save remaining list
kthNode.next = null;             // Cut current group
reverseLinkedList(temp);         // Reverse group

if (temp == head) {
    head = kthNode;               // First group
} else {
    prevLast.next = kthNode;      // Later groups
}

prevLast = temp;                 // New last node
temp = nextNode;                 // Next group

```

9. Why prevLast Is Needed

After reversal, the old first node becomes the last node of that group. That node must connect to the next reversed group.

```

1 → 2 → 3 | 4 → 5 → 6
3 → 2 → 1    6 → 5 → 4
    ↑
    prevLast = 1
3 → 2 → 1 → 6 → 5 → 4

```

10. First Group vs Later Groups

First group: the K-th node becomes the new overall head.

```

1 → 2 → 3
reverse
3 → 2 → 1
↑
head = 3

```

Later group: head does not change; prevLast connects the previous group to the current group's new head.

```

3 → 2 → 1      4 → 5 → 6

```

```

1.next = 6

```

```

3 → 2 → 1 → 6 → 5 → 4

```

11. Dry Run — k = 3

Input: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10

Group 1: temp=1, kthNode=3, nextNode=4. Cut, reverse to 3→2→1, update head=3, set prevLast=1, temp=4.

Group 2: temp=4, kthNode=6, nextNode=7. Reverse to 6→5→4, connect 1→6, set prevLast=4, temp=7.

Group 3: temp=7, kthNode=9, nextNode=10. Reverse to 9→8→7, connect 4→9, set prevLast=7, temp=10.

Remainder: only node 10 remains, so getKthNode returns null. Attach it unchanged and stop.

```

Final:
3 → 2 → 1 → 6 → 5 → 4 → 9 → 8 → 7 → 10

```

12. Edge Cases

- head == null → return null.
- k == 1 → list stays unchanged.
- size < k → no reversal.
- size == k → whole list reverses.
- size not divisible by k → final smaller group stays unchanged.
- one node → unchanged.

13. Time Complexity — Clearly Explained

For each complete group, getKthNode walks through at most k nodes and reverseLinkedList walks through those same k nodes. Across all groups, the total number of node visits is proportional to n. Therefore the overall time is **O(n)**.

```

Finding K-th nodes across groups → O(n)
Reversing all complete groups    → O(n)
Pointer connections              → O(n) overall

```

```

Total Time Complexity            → O(n)

```

14. Space Complexity — Clearly Explained

No array, map, stack, or extra linked-list nodes are used. Only a constant number of pointer variables are used, and the existing links are changed in place. Therefore auxiliary space is **O(1)**.

```

Extra data structures → none
Pointer variables     → constant

```

```

Total Auxiliary Space → O(1)

```

15. Recursion Rule for Future PDFs

This particular K-group solution is iterative, so there is no recursive call stack. For any future recursive DSA problem, these notes should additionally contain: **intuition** → **base case** → **recursive call** → **return/backtracking** → **dry run of calls and returns** → **complexity including recursion-stack space**.

16. Quick Revision

Find K → save next → cut → reverse → connect
→ remember previous last → move to saved next

Complete K-group: reverse
Incomplete final group: unchanged

Time: $O(n)$
Auxiliary Space: $O(1)$

17. Final Mental Picture

Original:

1 → 2 → 3 | 4 → 5 → 6 | 7 → 8 → 9 | 10

Result:

3 → 2 → 1 → 6 → 5 → 4 → 9 → 8 → 7 → 10

The key sentence to remember: **Find K → save next → cut → reverse → connect → remember last → move to saved next.**